MÓDULO PROFESIONAL: ENTORNOS DE DESARROLLO (CÓDIGO 0487)

Unidad 2: Instalación y usos de entornos integrados de desarrollo (IDE)

PROFESORADO Y CONTACTO

Profesor: Jorge Cabranes Azcona | Email: jca920@educa.madrid.org

UNIDAD 2: INSTALACIÓN Y USOS DE ENTORNOS INTEGRADOS DE DESARROLLO (IDE)

CONTENIDOS

- Introducción
- Funciones de un entorno de desarrollo
- Principales entornos integrados de desarrollo
- Instalación de entornos integrados de desarrollo
- Uso básico y configuración de un entorno de desarrollo
- Creación, compilación y generación de ejecutables

Introducción

En el proceso de desarrollo de software, contar con las herramientas adecuadas es fundamental para garantizar la eficiencia, calidad y productividad. Aquí es donde entran en juego los Entornos de Desarrollo Integrados (IDE, por sus siglas en inglés). **Un IDE es un conjunto de herramientas que permiten a los desarrolladores escribir, probar y depurar sus aplicaciones de forma más eficiente, todo desde una única interfaz gráfica**.

Los entornos de desarrollo **son utilizados en la fase de codificación del ciclo de vida de software** independientemente del modelo que se utilice. Existen entornos de desarrollo para un solo lenguaje o para múltiples lenguajes.

El uso de un entorno de desarrollo no solo facilita la escritura del código, sino que también ayuda a gestionar grandes proyectos, automatizar tareas repetitivas y detectar errores en tiempo real. Esto **se traduce en un ahorro significativo de tiempo y una mayor calidad en el producto final**.

Funciones de un Entorno de Desarrollo

Las **principales funciones** de un entorno de desarrollo se dividen en varias categorías:

- **Editor de código:** El corazón de cualquier entorno de desarrollo es el editor de código. Aquí es donde los programadores **escriben y editan el código fuente de sus programas**. Los editores de código modernos incluyen características como el resaltado de sintaxis, que facilita la identificación de palabras clave y estructuras del lenguaje, y el autocompletado, que sugiere posibles comandos a medida que se escribe.

- **Compilador/Enlazador/Interpretador:** Dependiendo del lenguaje de programación que se esté utilizando, el IDE incluye un compilador o interpretador. Estos **se encargan de traducir el código fuente en un lenguaje que la máquina pueda entender**. El compilador convierte el código en un archivo ejecutable, mientras que el interpretador ejecuta el código directamente, línea por línea.

Funciones de un Entorno de Desarrollo

- **Compilador:** En los lenguajes compilados, el compilador **traduce el código fuente a código máquina o a un formato intermedio que la máquina puede entender**. Este proceso se realiza en fases: análisis léxico, sintáctico, semántico y finalmente la generación de código.
- **Enlazador (Linker):** Es una pieza intermedia, en lenguajes como C o C++, que trabaja después de la compilación, pero antes de que el programa se ejecute. **Su función es unir los diferentes módulos o archivos objeto generados por el compilador y las bibliotecas externas** necesarias para crear el ejecutable final.
- **Interpretador:** En lenguajes interpretados, como Python, el código fuente no se convierte directamente en un ejecutable. En su lugar, el interpretador **lee el código línea por línea y lo ejecuta en tiempo real.**

- **Depurador (Debugger):** Un depurador es una herramienta esencial que permite a los desarrolladores **identificar y corregir errores en su código**. Ofrece la posibilidad de ejecutar el programa paso a paso, examinar el estado de las variables en tiempo real y localizar la línea exacta donde se produce un fallo.

Funciones de un Entorno de Desarrollo

- **Control de versiones:** Los IDE modernos integran sistemas de control de versiones como Git. Estas herramientas permiten a los desarrolladores **gestionar los cambios en su código, colaborar con otros programadores y mantener un historial de las diferentes versiones** del proyecto. De esta forma, si ocurre algún error o conflicto, es posible volver a versiones anteriores del código sin pérdida de información.
- **Gestor de proyectos:** El entorno de desarrollo también facilita la gestión del proyecto, permitiendo **organizar el código en diferentes archivos y carpetas**, **agregar bibliotecas externas y definir rutas de ejecución**. Todo esto asegura que el código sea modular, fácil de mantener y escalable.
- **Automatización de tareas:** Algunos IDE permiten la automatización de tareas comunes, como la compilación del código, la ejecución de pruebas unitarias o la generación de informes. Esto **reduce el esfuerzo manual y minimiza la posibilidad de errores** humanos.

Funciones de un Entorno de Desarrollo

- **Simuladores y Emuladores:** Para el desarrollo de software que va a ser ejecutado en diferentes dispositivos o sistemas operativos, los IDE ofrecen simuladores y emuladores. Estas herramientas permiten a los desarrolladores **probar sus aplicaciones en un entorno controlado sin necesidad de tener acceso físico al hardware**.
- **Integración con herramientas externas (Plugins):** Los entornos de desarrollo pueden conectarse a otras herramientas y servicios que mejoran el flujo de trabajo, como bases de datos, plataformas de despliegue continuo o servidores de prueba.

Por tanto, un entorno de desarrollo no solo es una herramienta que facilita la escritura de código, sino que **es un componente clave que integra múltiples funcionalidades para optimizar todo el proceso de desarrollo**. Entender cómo utilizar todas estas herramientas de manera eficaz es esencial para cualquier programador profesional.

Principales entornos de desarrollo

En la actualidad, muchos IDEs están disponibles en **varias plataformas como Windows, Linux y macOS**. Esta flexibilidad permite a los desarrolladores elegir su entorno preferido sin preocuparse por el sistema operativo. Además, muchos de estos entornos ofrecen extensiones y plugins que permiten integrar soporte para prácticamente cualquier lenguaje o tecnología. La elección entre un entorno de desarrollo libre o propietario depende de múltiples factores, como el lenguaje de programación, el tipo de proyecto y el presupuesto disponible.

Entornos de desarrollo libres

Los entornos de desarrollo integrados (IDE) libres son aquellos **disponibles bajo una licencia de uso público**, lo que significa que no requieren pagar por ellos. Además, la mayoría de estos entornos se distribuyen como software de código abierto, permitiendo a los desarrolladores modificar y distribuir el código fuente. Estos IDEs **son populares por su accesibilidad y flexibilidad, y ofrecen soporte para una variedad de lenguajes de programación**.

Principales entornos de desarrollo

No obstante, **la elección del IDE depende mucho del lenguaje y del tipo de proyecto**. Cada lenguaje y entorno tiene IDEs más adecuados o populares entre su comunidad de desarrolladores.

Principales entornos de desarrollo libres:

IDE	Lenguajes soportados	URL
Visual Studio Code	Soporta múltiples lenguajes mediante extensiones	code.visualstudio.com
Eclipse	Ada, C/C++, Java, JavaScript, PHP, Python, etc.	eclipse.org
NetBeans	C/C++, Java, JavaScript, PHP, HTML5, etc.	netbeans.org
IntelliJ IDEA Community	Java, Kotlin, Groovy, Scala, Python, etc.	jetbrains.com
CodeLite	C/C++, PHP, Node.js	codelite.org
PyCharm Community	Python, HTML, CSS, JavaScript	jetbrains.com/pycharm

Principales entornos de desarrollo

Entornos de desarrollo propietarios

Los entornos de desarrollo propietarios **requieren una licencia de pago para acceder a todas sus funcionalidades**, aunque algunos ofrecen versiones limitadas de forma gratuita.

Principales entornos de desarrollo propietarios:

IDE	Lenguajes soportados	URL
IntelliJ IDEA Ultimate	Java, Groovy, Scala, Kotlin, Python, Ruby, SQL, etc.	jetbrains.com
Microsoft Visual Studio	C++, C#, Visual Basic, Python, HTML, JavaScript, TypeScript, etc.	visualstudio.microsoft.com
CLion	C, C++, Rust	jetbrains.com/clion
PyCharm Professional	Python, HTML, CSS, JavaScript, SQL	jetbrains.com/pycharm
C++ Builder	C++	embarcadero.com
Qt Creator	C++ con el framework Qt	qt.io

Principales entornos de desarrollo

Actualmente, el IDE más utilizado en muchos ámbitos es **Visual Studio Code (VS Code)**, gracias a su capacidad de ser adaptado a múltiples lenguajes y proyectos, **especialmente en desarrollo web** y aplicaciones ligeras.

IntelliJ IDEA es el IDE **preferido por muchos desarrolladores de Java**, sin embargo, **Eclipse** sigue siendo una **opción poderosa**, especialmente en entornos corporativos. **NetBeans**, mantenido por Apache, sigue siendo utilizado, pero **ha perdido relevancia** frente a los IDEs mencionados.

Microsoft Visual Studio, sigue siendo líder en el desarrollo de aplicaciones en C++, C# y .NET, especialmente en plataformas Windows.

Instalación de entornos integrados de desarrollo

A lo largo de esta unidad, **aprenderéis a instalar** y configurar tres de los principales entornos de desarrollo integrados utilizados actualmente: **Eclipse, IntelliJ IDEA y Visual Studio Code**. Estos IDEs son ampliamente utilizados por desarrolladores profesionales para trabajar en diferentes lenguajes y proyectos.

Para la instalación de los IDEs, **utilizaremos máquinas virtuales (VM),** ya que ofrecen mayor flexibilidad, especialmente si queremos trabajar en distintos sistemas operativos o entornos. En este sentido, instalaremos algunos IDEs en **Windows** y otros en una máquina virtual con **Ubuntu Desktop**, lo que permitirá que os familiaricéis con ambos sistemas operativos. De esta manera, podréis comparar las diferencias entre trabajar en un entorno Linux y Windows.

Comenzaremos con la **instalación de Eclipse**, para la cual os proporcionaré en el aula virtual dos prácticas con una guía detallada paso a paso. Más adelante, subiré otras prácticas para que realicéis de manera autónoma la **instalación de IntelliJ IDEA y Visual Studio Code**, aplicando los conocimientos adquiridos durante la primera instalación.

Instalación de entornos integrados de desarrollo

Eclipse: Visión global

Eclipse es uno de los entornos de desarrollo integrado (IDE) más reconocidos, especialmente cuando hablamos de programar en Java. permitiendo desarrollar **aplicaciones en múltiples lenguajes** como C/C++, Python, JavaScript, PHP, TypeScript, y muchos otros, gracias a la instalación de plugins (módulos complementarios).

Debido a esta flexibilidad, Eclipse es una herramienta muy popular entre desarrolladores que trabajan en **proyectos con múltiples lenguajes de programación**, ya que pueden utilizar **un solo entorno sin necesidad de cambiar** entre diferentes herramientas. Además, es **altamente personalizable**, permitiendo que los usuarios creen un entorno de trabajo a su medida.

Hasta 2015, Eclipse fue el IDE oficial utilizado para desarrollar aplicaciones Android. Sin embargo, Google decidió lanzar su propio IDE, Android Studio, basado también en Eclipse, pero más especializado en el desarrollo de aplicaciones móviles.

Instalación de entornos integrados de desarrollo

Características de Eclipse que facilitan y optimizan el proceso de desarrollo de software:

- **Atajos de código y autocompletar**: Pulsando Control + Espacio, Eclipse muestra sugerencias automáticas para completar el código mientras se escribe. Esta función no solo ahorra tiempo, sino que también ayuda a minimizar errores y facilita el descubrimiento de métodos o funciones.
- **Autocorrección de errores:** Eclipse analiza continuamente el código fuente en busca de errores. Si detecta algún problema sintáctico o inconsistencia, marca la línea con un subrayado rojo y ofrece sugerencias de corrección. En algunos casos, incluso puede corregir automáticamente los errores más simples.
- **Herramientas de refactorización:** Permiten realizar modificaciones al código de manera sencilla, como renombrar variables o métodos, mover miembros de clases o modificar referencias en el código. Estas herramientas aseguran que los cambios se apliquen de manera consistente en todo el proyecto.

Instalación de entornos integrados de desarrollo

- **Comparación de archivos:** Si se tienen dos versiones de un archivo de código, Eclipse permite compararlos en paralelo para identificar diferencias y trabajar con ambos simultáneamente.
- **Formateo de código:** Eclipse ayuda a mantener un código limpio y legible. Puedes configurar el formato que prefieras, establecer sangrías, coloración de sintaxis, y otras características que faciliten la lectura y el mantenimiento del código.

Instalación de Eclipse

Como en el módulo de programación estáis **aprendiendo a programar en java**, vamos a **instalar el Eclipse de Java estándar** y, en este sentido, en el siguiente apartado se explica qué paquetes básicos debemos instalar para desarrollar en Java.

Instalación de entornos integrados de desarrollo

Entorno necesario para desarrollar en Java

Para desarrollar y ejecutar programas en Java, necesitamos configurar un entorno que incluya dos componentes fundamentales: el **JRE** y el **JDK.** A continuación, explicamos cada uno de estos elementos y sus funciones.

- **JRE (Java Runtime Environment):** Es el software necesario para ejecutar aplicaciones desarrolladas en Java. **Es lo que instala cualquier usuario final que desee usar aplicaciones Java en su máquina**. Incluye dos componentes principales:
  - **JVM (Java Virtual Machine):** La Máquina Virtual de Java es un programa que actúa como un **intérprete para ejecutar el bytecode de Java**. Cuando se compila un programa Java, no se traduce directamente a código máquina específico de un procesador, sino a un formato intermedio llamado bytecode. La JVM es responsable de interpretar este bytecode y ejecutarlo en el sistema operativo subyacente. Su mayor ventaja es que permite que el mismo programa Java pueda ejecutarse en diferentes plataformas, como Windows, Linux o macOS, sin necesidad de recompilar.

Instalación de entornos integrados de desarrollo

- **Biblioteca estándar:** El JRE también incluye la biblioteca estándar de Java, un conjunto de clases predefinidas que permiten a los programas interactuar con el sistema operativo, gestionar archivos, trabajar con colecciones de datos, gestionar interfaces gráficas (GUI), redes y más. Estas clases hacen que el desarrollo en Java sea más eficiente, ya que no es necesario programar estas funcionalidades desde cero.

- **JDK (Java Development Kit):** Es un paquete más amplio que incluye todo lo **necesario para desarrollar programas en Java**. **Es lo que deben instalar los programadores**, ya que no solo permite ejecutar programas, sino también escribir y compilar código Java. Además del JRE, el JDK incluye las siguientes Herramientas de desarrollo:

  - **javac:** El compilador de Java, que transforma el código fuente (escrito en un archivo .java) en bytecode (archivo .class). Este bytecode es lo que luego interpreta la JVM para ejecutar el programa.
  - **jar:** Una herramienta que permite empaquetar múltiples archivos .class y otros recursos (como imágenes o archivos de configuración) en un único archivo JAR (Java ARchive). Los archivos JAR son muy útiles para distribuir aplicaciones Java.

Instalación de entornos integrados de desarrollo

- **javadoc:** Un generador de documentación que permite crear documentación a partir de los comentarios del código fuente. Es una herramienta fundamental para generar la documentación técnica que explica cómo usar las clases y métodos de una aplicación.

Resumiendo, el **JRE es necesario para ejecutar programas Java**, Incluye la JVM (que interpreta y ejecuta el bytecode) y la biblioteca estándar (clases y métodos prediseñados para facilitar el desarrollo), y el **JDK es necesario para desarrollar programas en Java**. Además de contener el JRE, incluye herramientas como javac (para compilar), jar (para empaquetar) y javadoc (para generar documentación).

Instalación de entornos integrados de desarrollo

Prácticas

- Práctica 2.1- Instalación de Eclipse IDE en una MV con Ubuntu Desktop
- Práctica 2.2- Instalación de IntelliJ IDEA en una MV con Windows

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Perspectivas en Eclipse

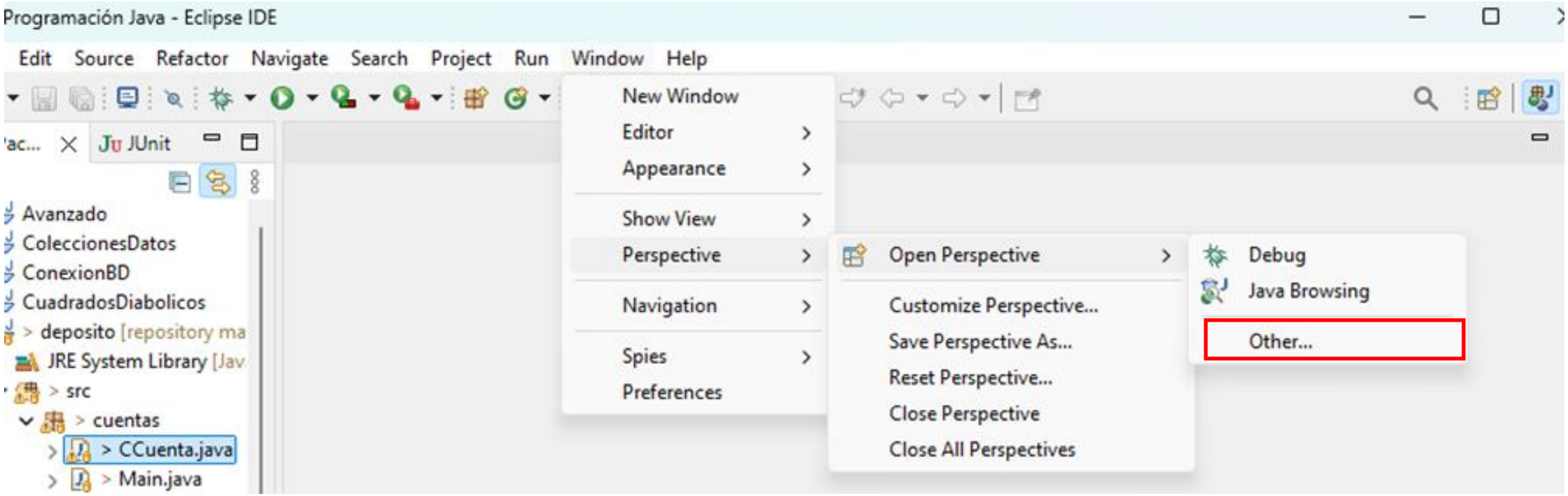
Dependiendo de la tarea en la que estemos trabajando en cada momento, demandaremos de Eclipse unos servicios u otros: desarrollar código en diferentes lenguajes (java, javascript, ..), depurar el programa, implementar plugins ... Para cada tarea, Eclipse ofrece un interfaz adaptado a las necesidades. Esto es lo que se llama perspectiva. Es decir, **una perspectiva es una configuración personalizada de ventanas y herramientas en Eclipse**.

Puedes cambiar de perspectiva desde el menú **Ventana > Perspectiva**.

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Perspectivas en Eclipse



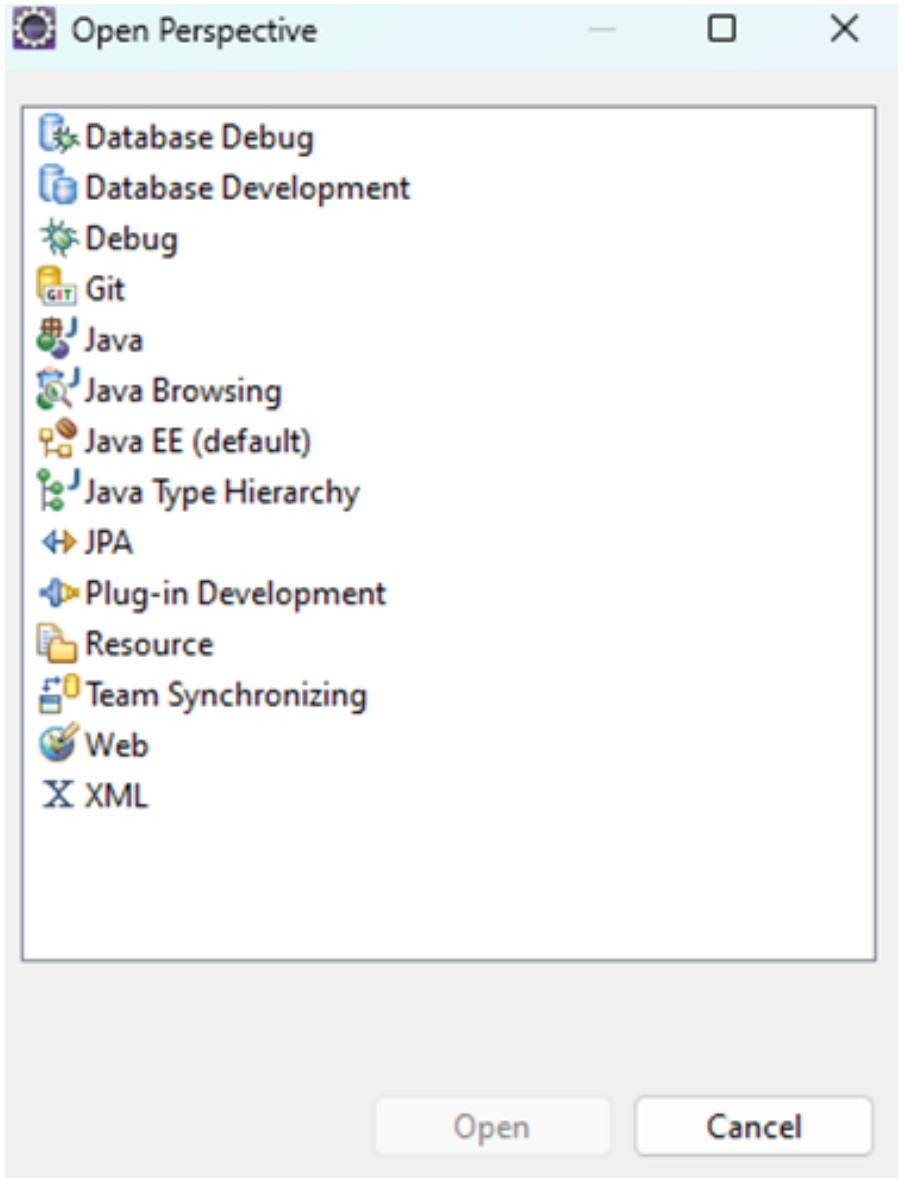
- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Perspectivas en Eclipse

Las **perspectivas disponibles** y que puedes ver en la figura anterior son las que Eclipse ofrece en su instalación base.

También es posible crear perspectivas personalizadas.

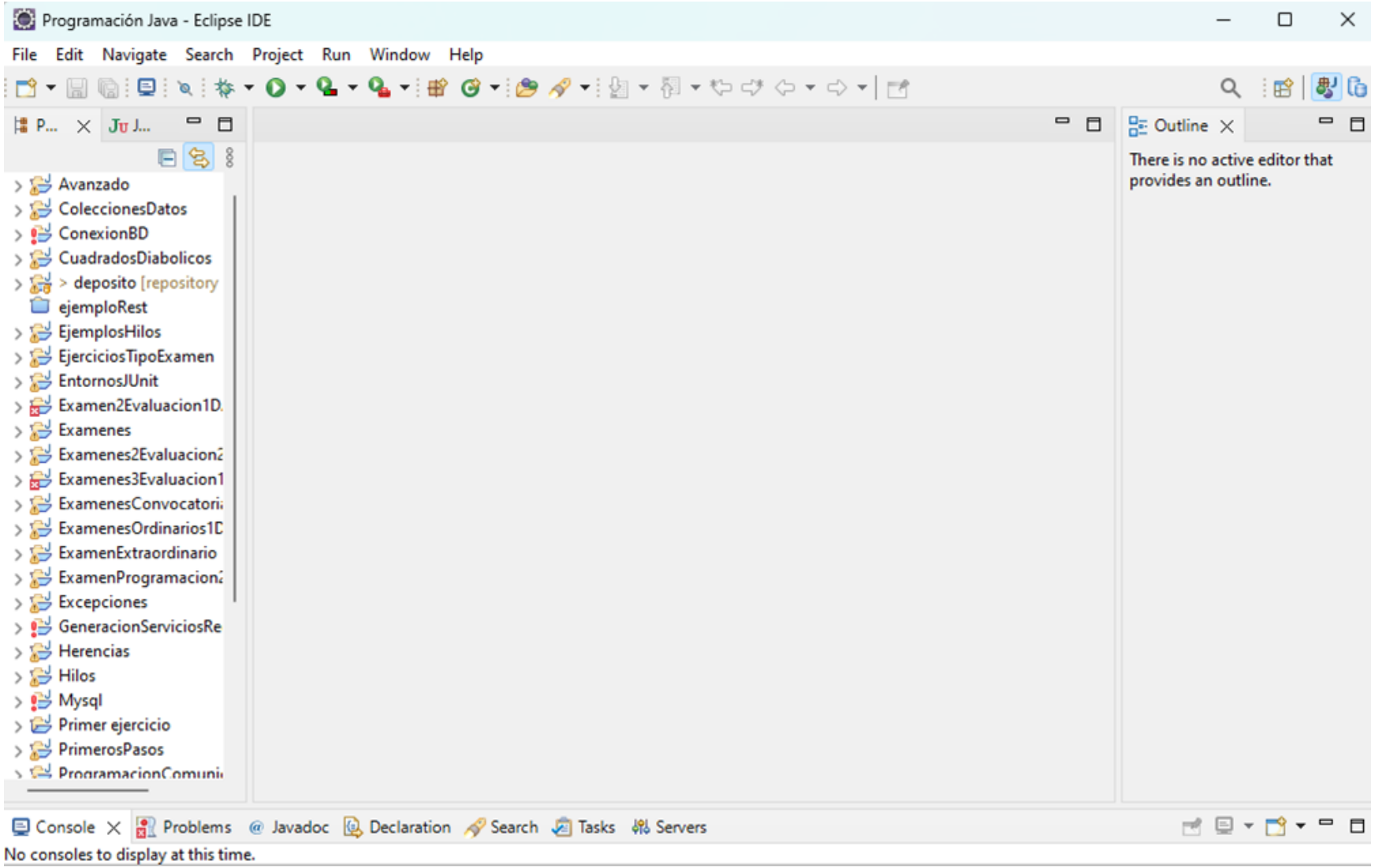


- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Perspectivas en Eclipse

Ejemplo: La **perspectiva Java** adapta las herramientas necesarias para desarrollar, depurar y gestionar proyectos Java.



- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Perspectivas en Eclipse

El **interfaz gráfico sigue un esquema común para la mayor parte de las perspectivas**:

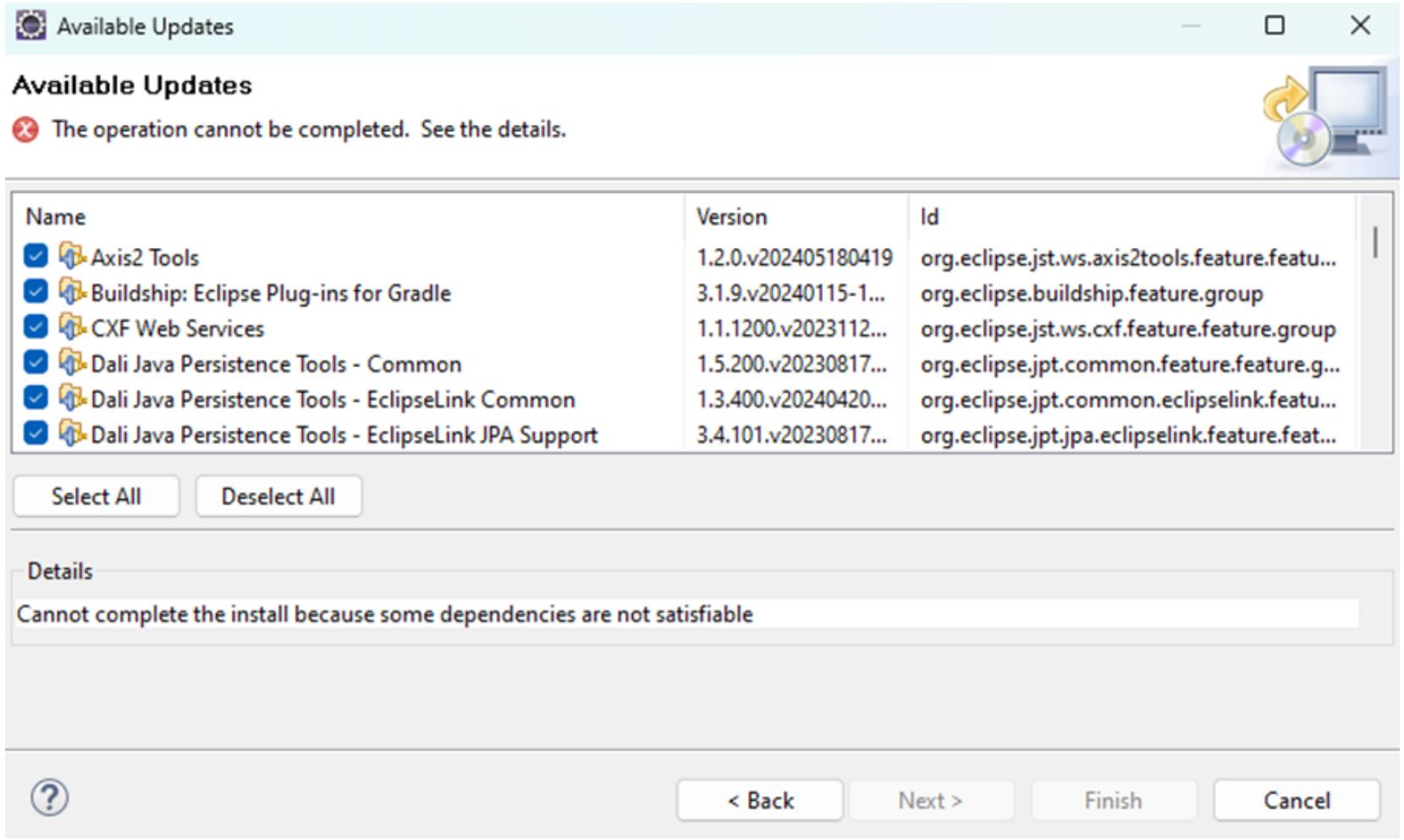
- **Zona de edición**. Espacio donde escribir el código de los programas. Es posible tener abiertos varios archivos simultáneamente. Éstos aparecerán organizados por pestañas.
- **Vista**. Se trata de un conjunto de ventanas secundarias que proporcionan funciones de apoyo. Cada perspectiva podrá tener sus propias vistas. Es posible mostrar u ocultar las vistas desde la opción de menú Ventana/Mostrar Vista.
- **Barra de herramientas**. Se trata de un acceso directo a las operaciones proporcionadas por Eclipse. Éstas están agrupadas por funcionalidades: abrir/crear nuevos proyectos, compilar, depurar, refactorizar, gestión de vistas y perspectivas ... Durante el curso iremos utilizando algunas de ellas.

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Actualización del IDE

Es importante mantener Eclipse actualizado para aprovechar nuevas características y correcciones. **Ayuda > Check for updates** para revisar actualizaciones disponibles.



- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

Eclipse **emplea módulos (plug-in)** para añadir funcionalidades a las ya existentes, se puede ver como "un esqueleto" que ofrece unas herramientas de partida, pero **es capaz de integrar diferentes paquetes adicionales que proporcionaran otros servicios requeridos por cada proyecto**.

Existen en la actualidad multitud de módulos disponibles que se pueden consultar en el "mercado de plugins" de Eclipse (https://marketplace.eclipse.org/).

Por tanto, **Eclipse Marketplace** es el lugar principal para descargar e instalar plugins (Ayuda > Eclipse Marketplace).

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

Ejemplo de **plugins útiles**:

- **Enhanced Class Decompiler**: Para descompilar archivos .class (bytecode) y ver el código fuente.
- **JBC** (Java ByteCode): Permite ver el bytecode de archivos .class.
- **UMLet**: Herramienta para crear diagramas UML.

Pasos para la **instalación de Plugins**:

- Ir a **Ayuda > Eclipse Marketplace**.
- Buscar el plugin.
- Instalar siguiendo las instrucciones y reiniciar Eclipse.
- Para gestionar/desinstalar plugins instalados, ir a la pestaña "Installed" en el mismo Marketplace. Aquí tendremos la posibilidad de solicitar su cambio-actualización o desinstalación.

- **Uso básico y configuración de un entorno de desarrollo**

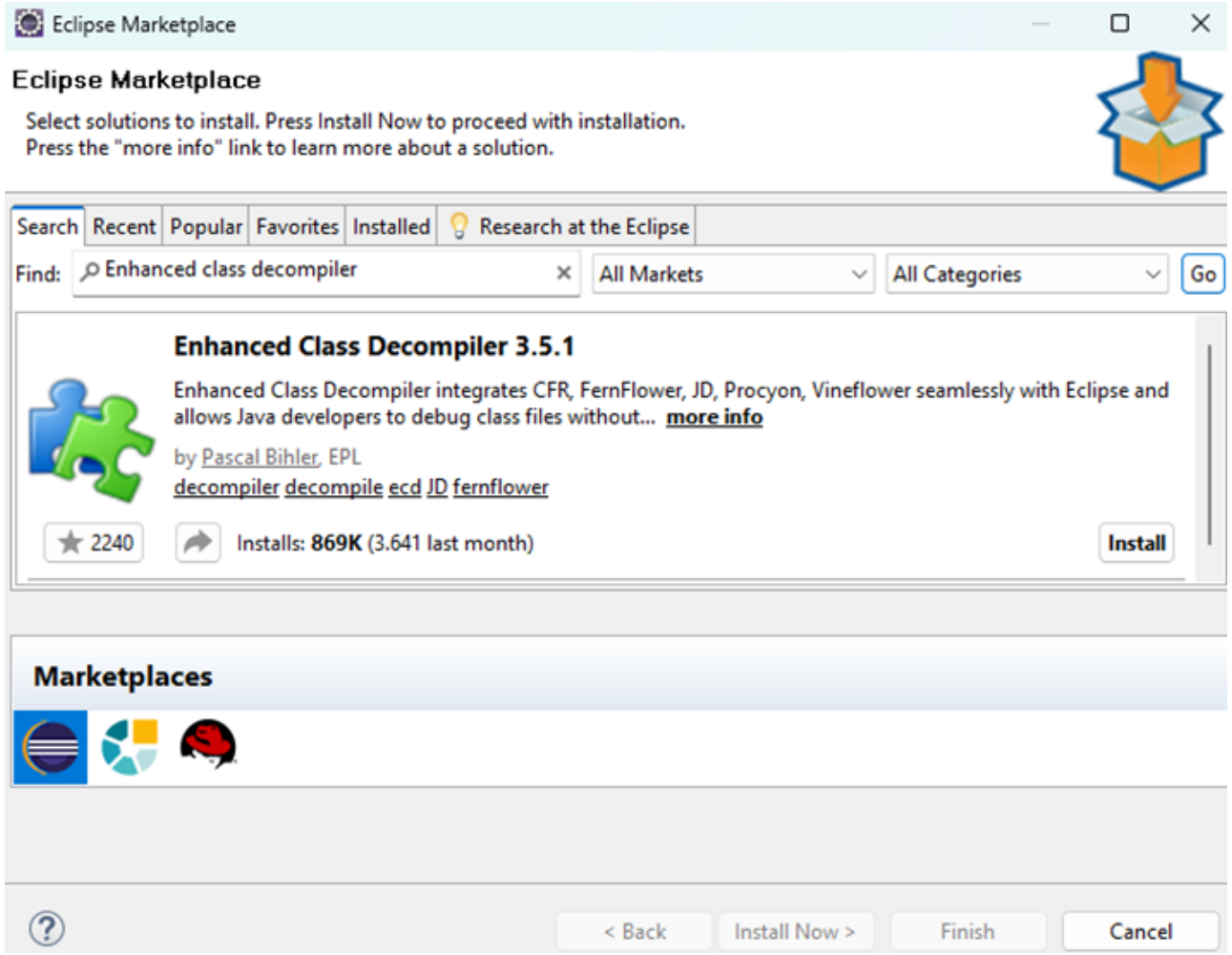
Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

Enhanced class decompiler

Una vez instalado y reiniciado Eclipse, seleccionar este plugin para mostrar el código descompilado de ficheros con extensión .class. Para ello, aplicaremos la siguiente configuración:

- Mostrar la ventana de preferencias desde la opción de menú "**Ventana/Preferencias**".
- Seleccionar "**General/Editores/Asociaciones de Archivo/*.class without source**".
- Marcar el módulo "**Class Decompiler Viewer**" y pulsar el boton "**Por defecto**". Finalizar el proceso aplicando los cambios.



- **Uso básico y configuración de un entorno de desarrollo**

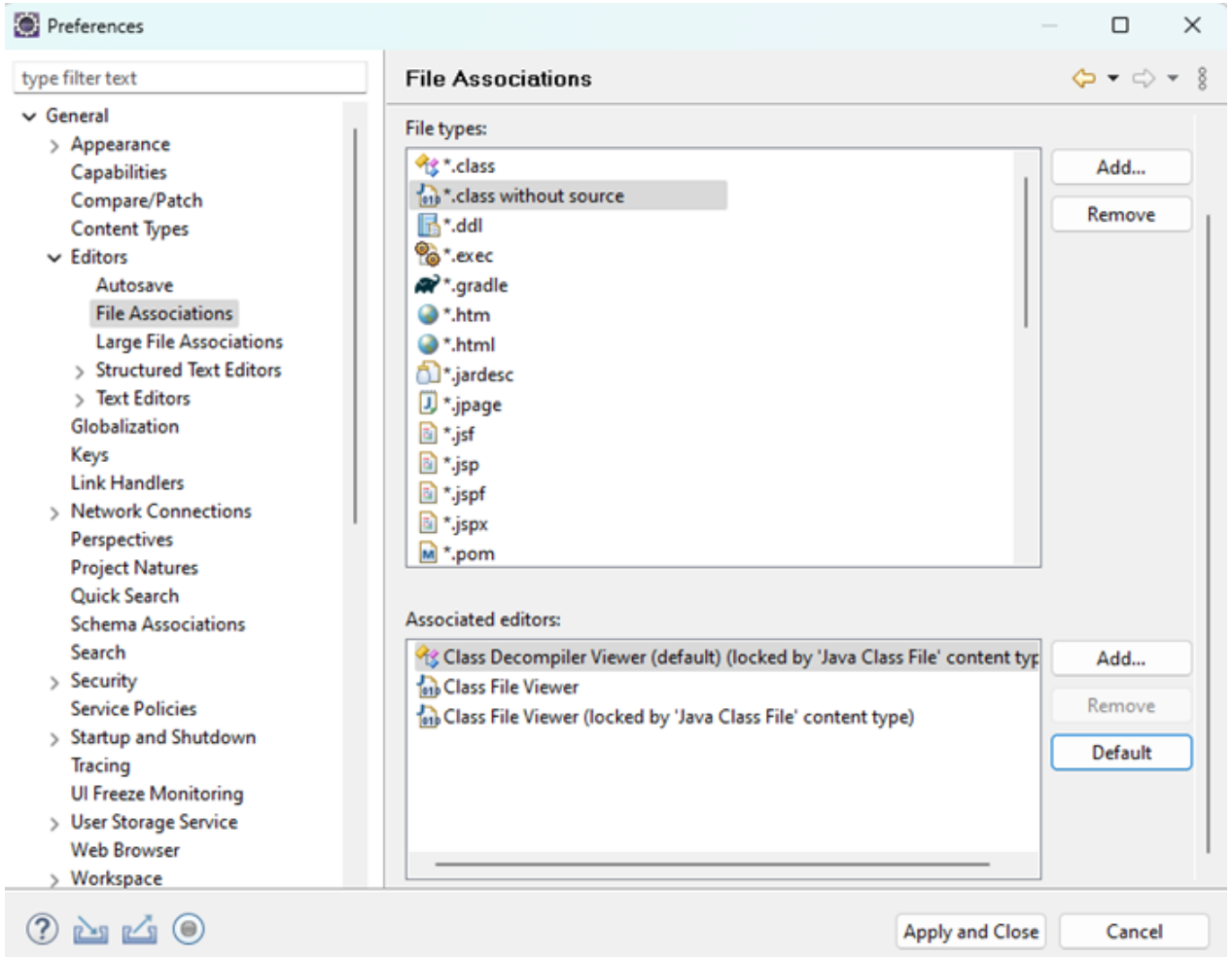
Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

Enhanced class decompiler

Para **comprobar su funcionamiento**, sobre alguna clase de los paquetes de Java (System por ejemplo) de la que no disponemos de su código fuente:

- Abre cualquier clase de tu proyecto que utilice una clase de la API de Java, como System. Mantén presionada la **tecla Ctrl** (o Cmd en macOS) y haz **clic en el nombre de la clase que deseas descompilar** (en este caso, System).
- Alternativamente, puedes hacer clic derecho sobre el nombre de la clase (como System) y **seleccionar Open Implementation u Open Declaration**.



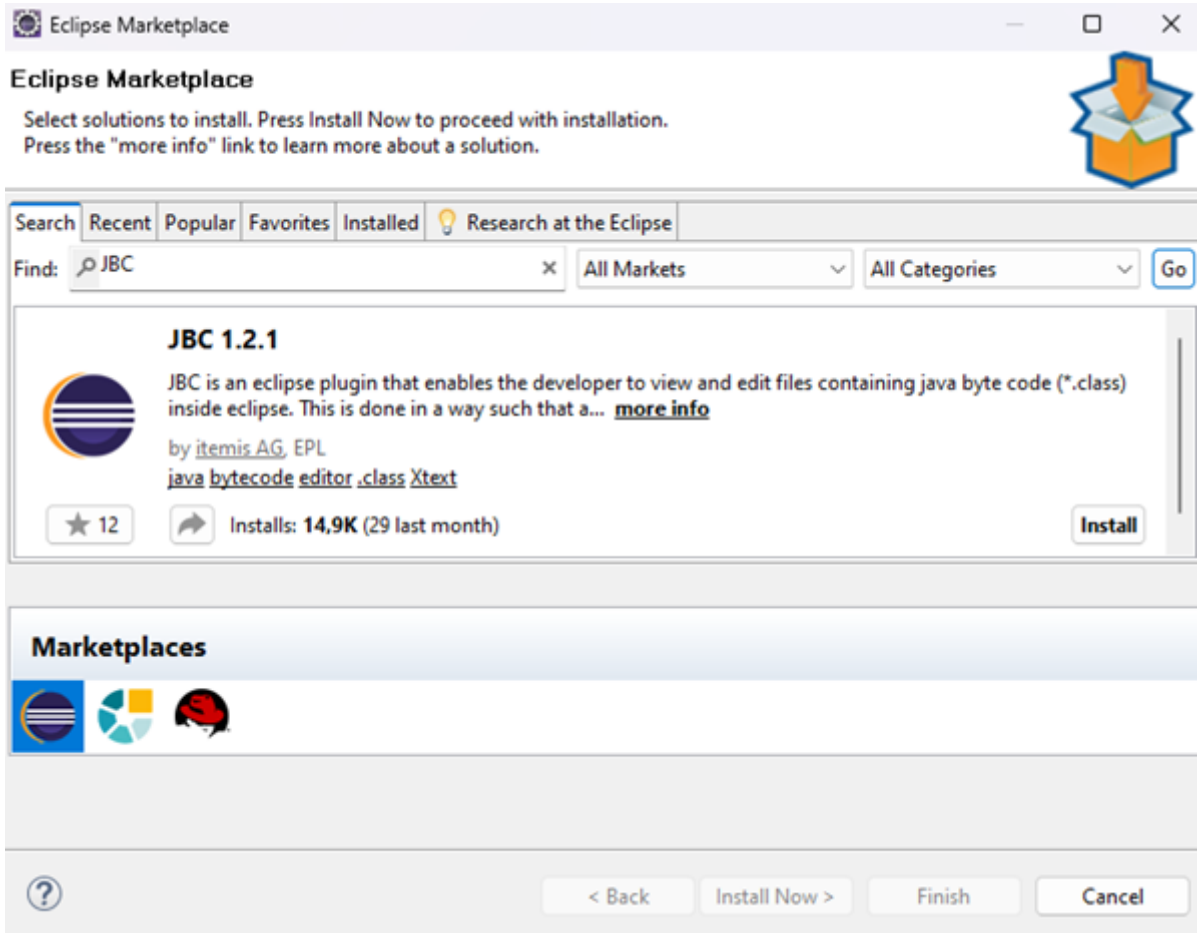
- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

JBC

JBC (Java ByteCode) permite ver la información compilada de un fichero .class desde el IDE. La instalación se hará nuevamente desde el MarketPlace de Eclipse, a través de la opción de menú "Ayuda/EclipseMarketPlace".



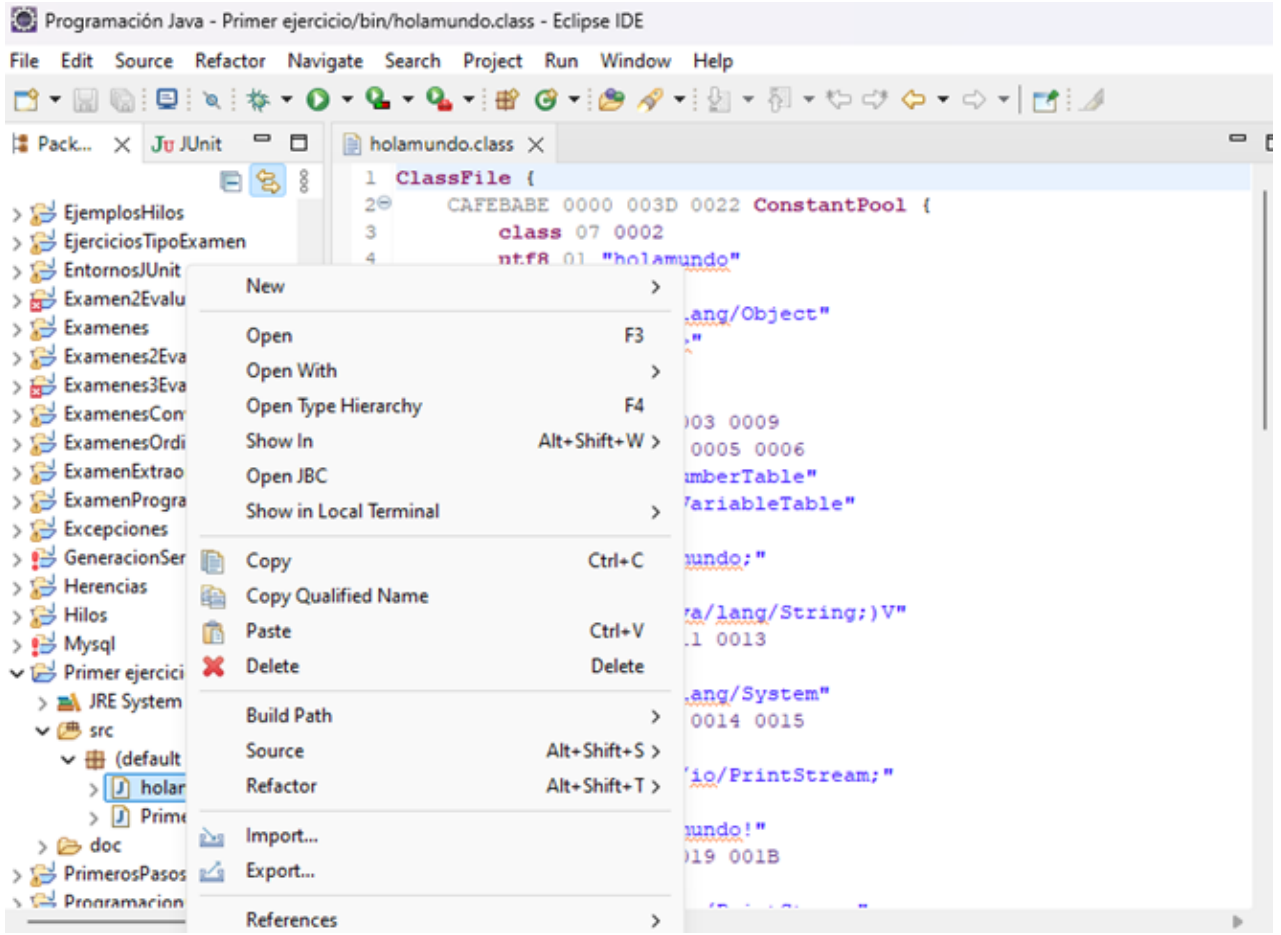
- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

JBC

Para **comprobar su funcionamiento**, pulsar con el botón derecho del ratón sobre un fichero con extensión .java en el "Explorador de paquetes" (vista disponible en la perspectiva Java). Seleccionar la opción Open JBC del menú contextual.



- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

UMLet

Durante el curso conoceremos una serie de diagramas de modelado útiles en la fases de análisis y diseño en los desarrollos software. Existe muchas herramientas que nos permite dibujar estos diagramas, una de ellas es **UMLet**.

UMLet está disponible tanto en formato stand-alone, como para ser integrada en otras aplicaciones. Existe un **plugin** de **UMLet** para **Eclipse**.

En esta ocasión utilizaremos una técnica distinta para añadir el plugin.

- **Descargar** de la url http://www.umlet.com/changes.htm el **plugin UMLet** para **Eclipse** (versión 15.0).
- **Descomprimir** y copiar el fichero **com.umlet.plugin_15.0.0.jar** en la ruta **plugins o dropings de Eclipse (*)**. Asegúrate que tiene permisos de lectura el fichero (comando chmod en **Linux**).
- **Reiniciar** Eclipse.
- Para iniciar **UMLet**, **crear un nuevo proyecto** desde la opción de menú "**Archivo/Nuevo/Otras/Otro/Umlet diagram**".

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

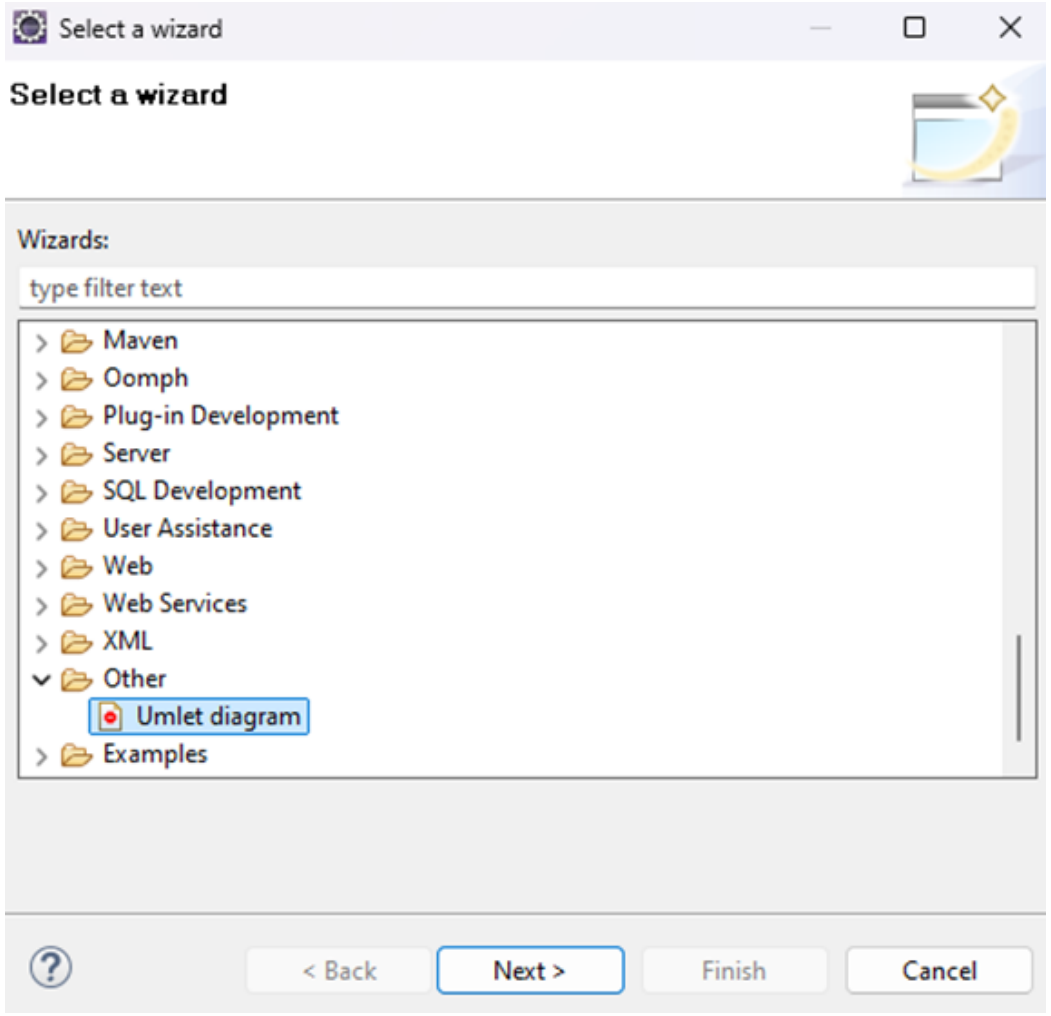
Instalación de complementos (plugins)

UMLet

Verificar la instalación del plugin:

Después de reiniciar, ve a **Help > About Eclipse IDE > Installation Details**.

Busca el plugin UMLet o cualquier referencia al archivo com.umlet.plugin_15.0.0.jar para verificar que está correctamente instalado.

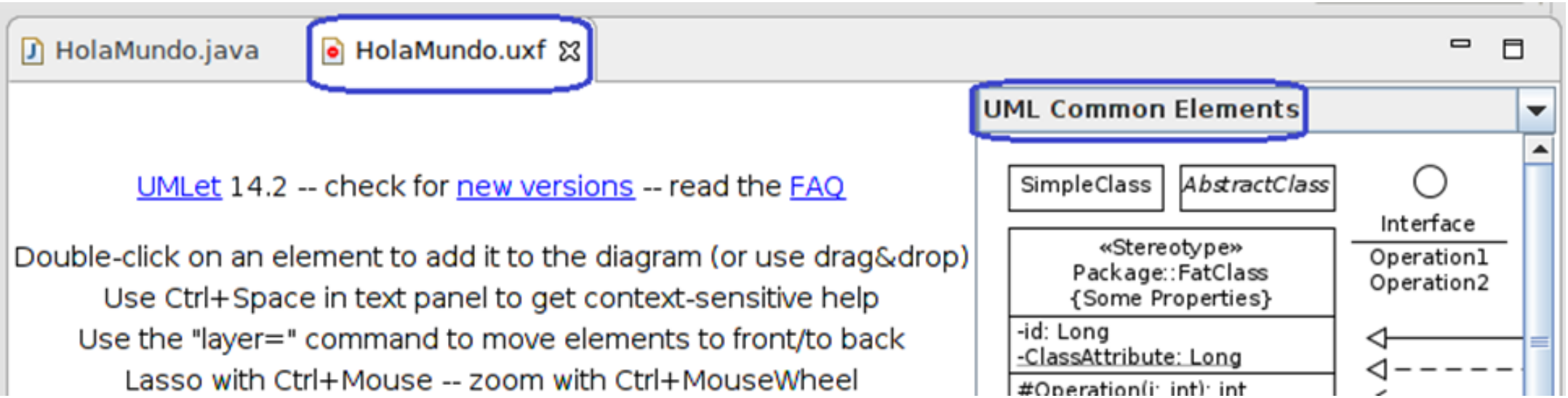


- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Instalación de complementos (plugins)

UMLet



Para desinstalar el plugin sólo tendrás que borrar el fichero .jar de la ruta plugins o dropings de Eclipse.

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Cambio de idioma en Eclipse

Eclipse permite cambiar el idioma mediante la instalación del paquete Babel.
Ayuda > Install New Software, buscar "Babel" y seleccionar el idioma deseado.

Añadir el repositorio de Babel

- En la ventana Install New Software, en la parte superior, encontrarás un campo llamado Work with:.
  - Haz clic en el menú desplegable y selecciona Add... para añadir un nuevo repositorio.
  - En la ventana emergente, añade el nombre y la URL del repositorio:
    - Name: Babel
    - Location: http://download.eclipse.org/technology/babel/update-site/latest/
    - Haz clic en OK para añadirlo

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

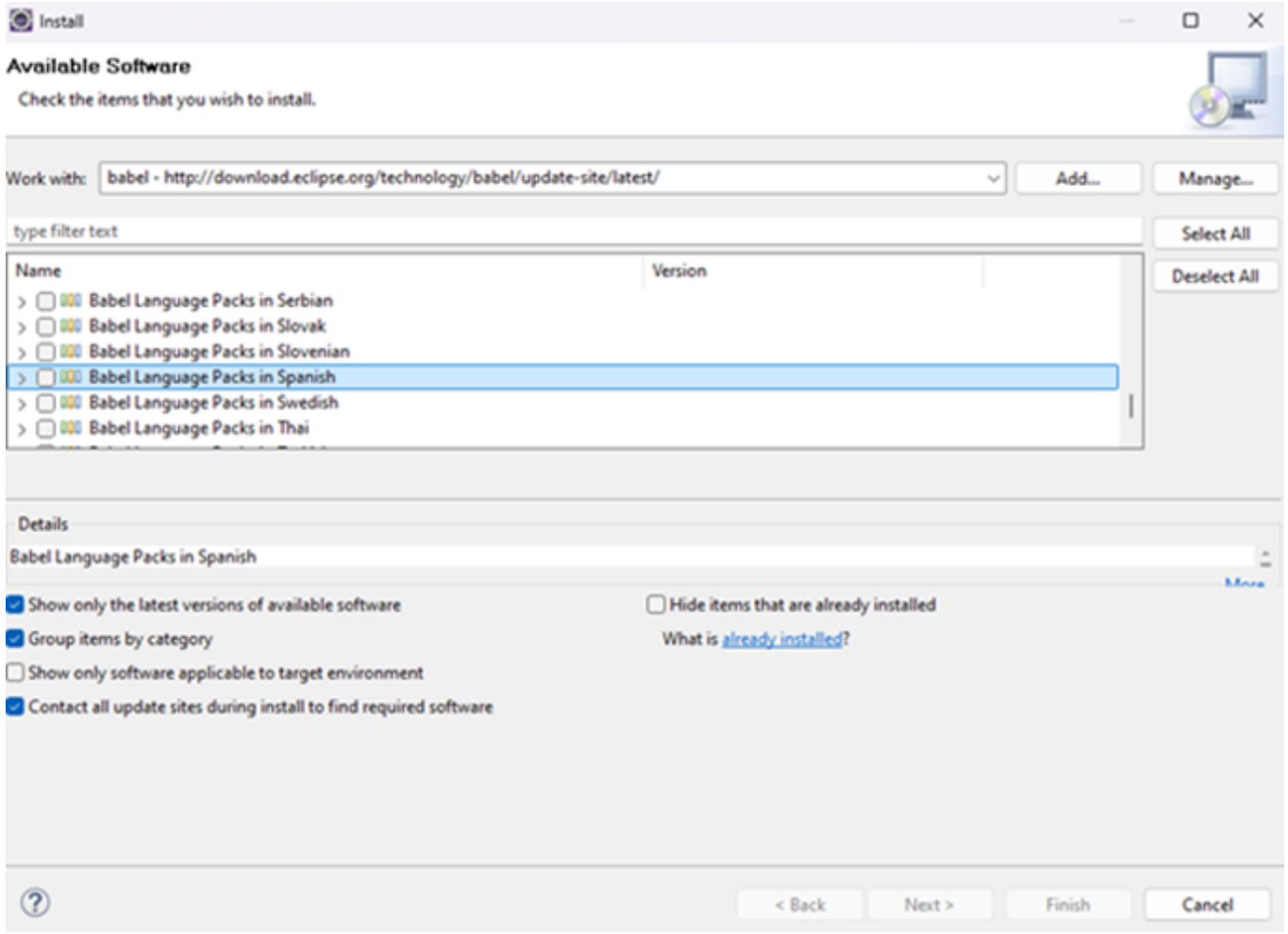
Cambio de idioma en Eclipse

- Buscar el paquete Babel. En la lista que aparece debajo de Work with, verás las opciones del repositorio Babel que acabas de agregar.
  - Expande las categorías que aparecen, seleccionando tu versión de Eclipse.
  - Busca el paquete de idioma que deseas instalar. Si deseas instalar Eclipse en español, selecciona Babel Language Packs in Spanish (es).
  - Marca la casilla correspondiente al idioma que deseas instalar y haz clic en Next.
- Reiniciar Eclipse

- **Uso básico y configuración de un entorno de desarrollo**

Uso y Configuración de Eclipse IDE

Cambio de idioma en Eclipse



Creación, compilación y generación de ejecutables

Crear, compilar y ejecutar un programa en Linux

Utilizando un editor de textos (ej.:nano) crear el fichero **Hola.java** con el siguiente código:

```
public class Hola {
	public static void main(String[ ] args){
		System.out.println("Hola");
	}
}
```

- Salva el archivo en alguna ruta de tu sistema de archivos. Ej.: "/home/usuario/".
- Abrir un terminal en Linux y moverse a la ruta donde se ha creado el fichero Hola.java.
- Ejecutar el comando **javac Hola.java** para compilar. Obtenemos el fichero Hola.class
- Interpretar Hola.class con el comando **java Hola**.

Creación, compilación y generación de ejecutables

Ejemplo práctico: Simulación de un semáforo

Imagina que un **profesor** de autoescuela quiere enseñar a un estudiante cómo funciona la secuencia de colores de un semáforo. Para hacerlo, en el aula tienen un **ordenador** que simula el semáforo, permitiendo consultar qué color está mostrando en cada momento. El profesor pregunta al **estudiante** qué color tiene el semáforo, y el estudiante, tras consultar el ordenador, responde.

Solución propuesta (hay muchas maneras diferentes de implementar una solución)

El problema se puede plantear utilizando tres actores principales u objetos:

- **Profesor**: quien hace la pregunta.
- **Estudiante**: quien responde tras consultar el ordenador.
- **Ordenador**: simula el semáforo, proporcionando el color actual.

Para cada uno de estos actores, se desarrollará una clase en Java. Al compilar el código, obtendremos un fichero por cada clase que se traducirá en un bytecode de Java (fichero .class), que será interpretado para su ejecución.

- ## Creación, compilación y generación de ejecutables

Ejemplo práctico: Simulación de un semáforo

Organización del proyecto

En Java, un proyecto se organiza en directorios y subdirectorios. En este caso, nuestro proyecto se llamará **semaforo**, y dentro de él tendremos:

Cada clase se almacenará en un fichero de código fuente (fichero .java), que implementa su funcionalidad. A continuación, se explica cada clase:

- **Creación, compilación y generación de ejecutables**

Ejemplo práctico: Simulación de un semáforo

Descripción de las clases

Clase clasecolor (clasecolor.java)

```
package principal;
import clases.profesor;

public class clasecolor {
	public static void main(String[] args) {
		profesor teacher = new profesor();
		String color = teacher.preguntacolor();
		System.out.println("La respuesta recibida es:" + color);
	}
}
```

- **Creación, compilación y generación de ejecutables**

Ejemplo práctico: Simulación de un semáforo

Descripción de las clases

Clase ordenador (ordenador.java)

```
package clases;
import java.util.Random;

public class ordenador {
    public ordenador() {}
    public String color(){
      Random randomGenerator = new Random();
            int randomInt = randomGenerator.nextInt(3);
            if(randomInt == 0){
                return "rojo";
            }else if(randomInt == 1){
                return "amarillo";
            }else {
                return "verde";
            }
    }
}
```

- **Creación, compilación y generación de ejecutables**

Ejemplo práctico: Simulación de un semáforo

Descripción de las clases

Clase persona (persona.java)

```
package clases;

public class persona {
     int Edad;
     String Nombre;
}
```

- **Creación, compilación y generación de ejecutables**

Ejemplo práctico: Simulación de un semáforo

Descripción de las clases

Clase estudiante (estudiante.java)

```
package clases;

public class estudiante extends persona{
	int Curso;

	public estudiante() {
		Edad=25;
		Nombre = "Luis";
		Curso = 1;
	}
	public void presentarse(){
	  System.out.println("Soy " + Nombre + " Alumno de " + Curso
	  + "º y tengo una edad de: " + Edad);
	}
	public String preguntacolor(){
		presentarse();
		ordenador mipc = new ordenador();
		return mipc.color();
	}
}
```

- **Creación, compilación y generación de ejecutables**

Ejemplo práctico: Simulación de un semáforo

Descripción de las clases

Clase profesor (profesor.java)

```
package clases;

public class profesor extends persona {
     public profesor() {
     }

     public String preguntacolor() {
            estudiante alumno = new estudiante();
            return alumno.preguntacolor();
     }
 }
```

Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Cuando trabajamos en un **proyecto Java**, todos sus recursos **se organizan dentro de un directorio en el sistema de archivos**. En este caso, el directorio del proyecto es ("/home/usuario/eclipse-workspace/EDSemaforo".

Dentro de este directorio, distribuimos los archivos del código fuente en diferentes **paquetes**, que **corresponden a subdirectorios**. Por ejemplo, los archivos ordenador.java, persona.java, profesor.java y estudiante.java están organizados en el paquete clases, mientras que el archivo clasecolor.java, que contiene la función principal (**main**), está en el paquete principal.
workspace/EDSemaforo" en nuestro caso).

```
jorge@Ubuntu-Client:~$ ls /home/jorge/eclipse-workspace/EDSemaforo/clases
estudiante.java   ordenador.java   persona.java   profesor.java

jorge@Ubuntu-Client:~$ ls /home/jorge/eclipse-workspace/EDSemaforo/principal
clasecolor.java
```

Creación, compilación y generación de ejecutables

Uso del Compilador de Java

En el código fuente de cada clase, es necesario especificar a qué paquete pertenece utilizando la instrucción **package**. Para usar las clases de un paquete en otro archivo, debemos importarlas con la instrucción **import**, excepto cuando las clases están en el mismo paquete.

Compilación de las Clases

Para compilar el proyecto, **comenzaremos compilando las clases que están en la carpeta clases** y que son referenciadas por la clase clasecolor. Es importante que las **variables de entorno** de Java estén **correctamente configuradas** (según lo trabajado en prácticas anteriores).

Desde la carpeta del proyecto ("/home/usuario/eclipse-workspace/EDSemaforo"), ejecutamos el siguiente comando para compilar todas las clases del paquete clases:

$ javac clases/*.java

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ javac clases/*.java
```

- ## Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Este comando invoca al compilador de Java (javac), que analiza los archivos fuente (.java) y **genera los archivos de bytecode correspondientes (.class).** Si el proceso de compilación se realiza correctamente, no aparecerá ningún mensaje en la terminal, pero podremos ver los archivos .class generados en la carpeta.

```
jorge@Ubuntu-Client:~$ ls /home/jorge/eclipse-
workspace/EDSemaforo/clases
estudiante.class  ordenador.class  persona.class  profesor.class
estudiante.java   ordenador.java   persona.java   profesor.java
```

Después de compilar las clases del paquete clases, procederemos a **compilar la clase principal (clasecolor),** que se encuentra en el paquete principal. Para hacerlo, ejecutamos el siguiente comando desde la misma carpeta del proyecto:

$ javac principal/clasecolor.java

- **Creación, compilación y generación de ejecutables**

Uso del Compilador de Java

El resultado será un archivo bytecode (clasecolor.class), que se podrá ejecutar con la Máquina Virtual de Java (JVM).

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ javac principal/clasecolor.java

jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ ls /home/jorge/eclipse-
workspace/EDSemaforo/principal
clasecolor.class  clasecolor.java
```

- # Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Ejecución del Programa

Una vez compiladas todas las clases necesarias, podemos **ejecutar el programa** utilizando el siguiente comando:

$ java principal.clasecolor

Este comando ejecutará el programa y mostrará el resultado en pantalla, que podría ser, por ejemplo, el color del semáforo según la lógica implementada.

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ java
principal.clasecolor
Soy Luis Alumno de 1 y  tengo una edad de: 25
La respuesta recibida es:amarillo
```

Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Empaquetar los Archivos .class en un Archivo .jar

Java nos permite empaquetar todos los archivos compilados (.class), junto con otros recursos, en un solo archivo comprimido con extensión .jar (Java Archive). **Este formato tiene la ventaja de ahorrar espacio y simplificar la distribución del proyecto**.

- **Creación del Archivo MANIFEST.MF**

  Para crear un archivo .jar, primero necesitamos un fichero llamado **MANIFEST.MF** que **especifique cuál es la clase principal del programa**. Este fichero debe ubicarse en una carpeta llamada META-INF (importante escribirlo en mayúsculas).

  El contenido del archivo MANIFEST.MF suele ser sencillo, como por ejemplo: Main-Class: principal.clasecolor

Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Esto indica que la clase principal que contiene el método main es clasecolor, dentro del paquete principal. Asegúrate de que haya un salto de línea después de esta última línea de texto.

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ mkdir -p META-INF
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ cd META-INF
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo/META-INF$ sudo nano MANIFEST.MF
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo/META-INF$ sudo cat MANIFEST.MF
Main-Class: principal.clasecolor
```

Creación del Archivo .jar

Una vez creado el archivo MANIFEST.MF, podemos empaquetar el proyecto en un archivo .jar utilizando el siguiente comando:

$ jar cmf META-INF/MANIFEST.MF semaforo.jar clases/*.class principal/*.class

Este comando crea un archivo comprimido (semaforo.jar) que contiene los archivos .class de los paquetes clases y principal, junto con el fichero MANIFEST.MF que indica la clase principal.

- **Creación, compilación y generación de ejecutables**

Uso del Compilador de Java

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ jar cmf META-INF/MANIFEST.MF
semaforo.jar clases/*.class principal/*.class

jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ ls -l
total 16
drwxrwxr-x 2 jorge jorge 4096 oct 17 11:00 clases
drwxrwxr-x 2 jorge jorge 4096 oct 17 12:07 META-INF
drwxrwxr-x 2 jorge jorge 4096 oct 16 19:37 principal
-rw-rw-r-- 1 jorge jorge 3110 oct 17 12:10 semaforo.jar
```

Para ejecutar el programa directamente desde el archivo .jar, podemos usar el siguiente comando:

$ java -jar semaforo.jar

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ java -jar semaforo.jar
Soy Luis Alumno de 1 y  tengo una edad de: 25
La respuesta recibida es:rojo
```

- **Creación, compilación y generación de ejecutables**

Uso del Compilador de Java

Además, el programa jar también permite ver el contenido de un archivo .jar sin descomprimirlo:

$ jar tf semaforo.jar

```
jorge@Ubuntu-Client:~/eclipse-workspace/EDSemaforo$ jar tf semaforo.jar
META-INF/
META-INF/MANIFEST.MF
clases/estudiante.class
clases/ordenador.class
clases/persona.class
clases/profesor.class
principal/clasecolor.class
```

Si necesitamos descomprimir el contenido del archivo .jar, simplemente usamos:

$ jar xf semaforo.jar

Es recomendable descomprimir en una carpeta separada para evitar confusión con los archivos originales.

- ## Creación, compilación y generación de ejecutables

Uso del Compilador de Java

Creación de un Script para Facilitar la Compilación

Para automatizar el proceso de compilación y empaquetado, podemos crear un **script llamado** **compilar.sh.** Este script debe contener los comandos necesarios para compilar el proyecto y crear el archivo .jar. El contenido del script podría ser el siguiente:

```
javac principal/clasecolor.java
jar cmf META-INF/MANIFEST.mf semaforo.jar clases/*.class principal/*.class
```

Para poder ejecutar el script, es necesario darle permisos de ejecución con el siguiente comando:

```
$ chmod +x compilar.sh
```

Luego, se puede ejecutar el script con:

```
$ ./compilar.sh
```

Este script compilará el archivo clasecolor.java y creará el archivo .jar del proyecto automáticamente.